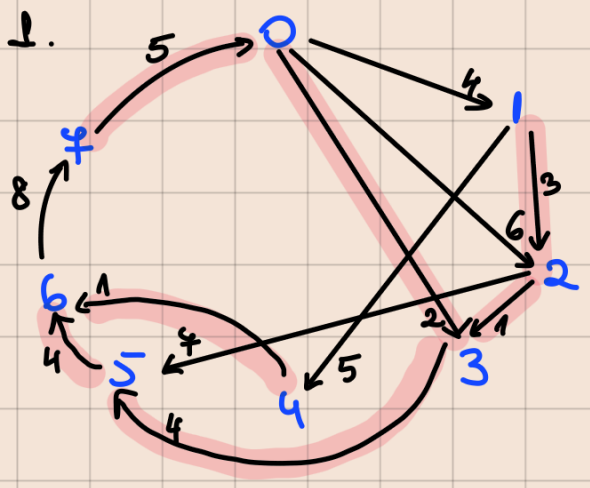
Construct minimum spanning tree using Prim's algorithm

1.



→ starting vertex = 2

Edges	Cost
0 - 1	4
0 - 2	6
0 - 3	2
1 - 2	3
1 - 4	5
2 - 3	1
2 - 5	7
3 - 5	4
4 - 6	1
5 - 6	4
6 - 7	8
7 - 0	5

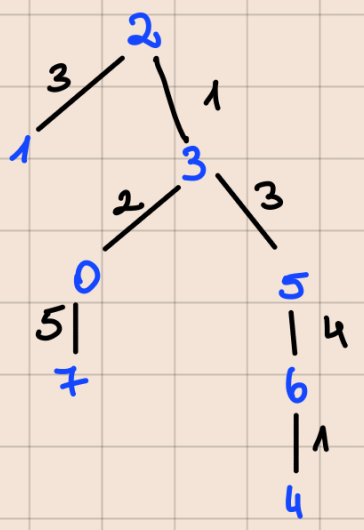
Sorted edges	
2 - 3	1
4 - 6	1
0 - 3	2
1 - 2	3
0 - 1	4
3 - 5	4
5 - 6	4
1 - 4	5
7 - 0	5
0 - 2	6
2 - 5	7
6 - 7	8

	node	total cost	neighbours	queue until now	add to queue:
it 1	2	0	{0, 1, 3, 5}	[ ]	(6, 0, 2) (3, 1, 2) (1, 3, 2) (7, 5, 2)
it 2	3	1	{0, 2, 5}	[(3, 1, 2), (6, 0, 2) (7, 5, 2)]	(2, 0, 3) (4, 5, 3)
it 3	0	3	{1, 2, 3, 7}	[(3, 1, 2), (4, 5, 3), (7, 5, 2), (6, 0, 2)]	(4, 1, 0) (5, 7, 0)

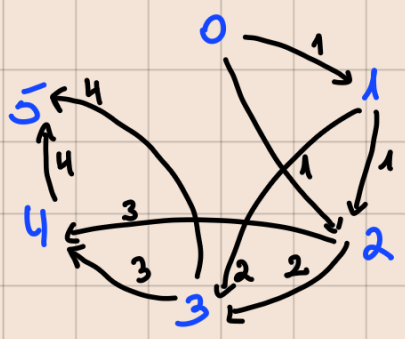
it 4	1	6	{0, 2, 4}	[(4,1,0), (4,5,3), (5,7,0), (6,0,2), (7,5,2)]	(5, 4, 1)
it 5	5	10	{2, 3, 6}	[(5,4,1), (5,7,0) (7,5,2) (6,0,2)]	(4, 6, 5)
it 6	6	14	{4, 5, 7}	[(5,4,1), (5,7,0), (7,5,2), (6,0,2)]	(1, 4, 6) (8, 7, 6)
it 7	4	15	{1, 6}	[(5,4,1), (5,7,0), (7,5,2), (6,0,2), (8,7,6)]	—
it 8	7	20	{0, 6}	[(6,0,2), (8,7,6), (7,5,2)]	—

⇒ minimum cost = 20

edges: 0 → 3, 1 → 2, 3 → 2, 4 → 6, 5 → 3, 6 → 5, 7 → 0

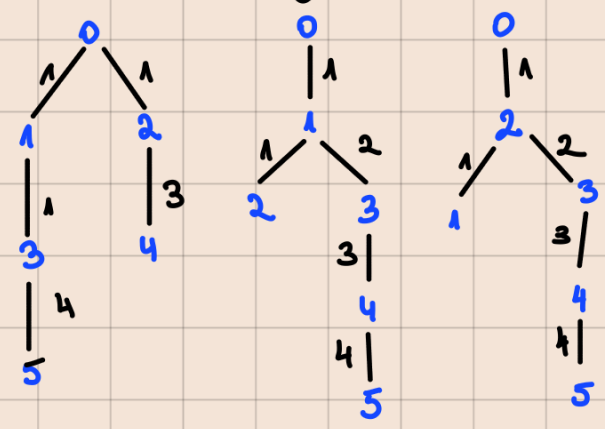


2. Small graph: 6 vertices, 9 edges



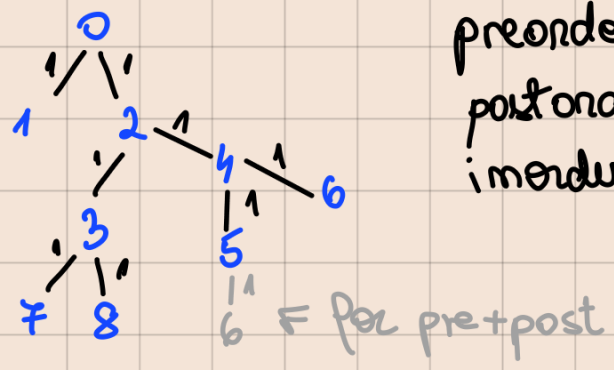
Edges	Cost
0-1	1
0-2	1
1-2	1
1-3	2
2-3	2
2-4	3
3-4	3
4-5	4
3-5	4

→ minimal graph cost: 11



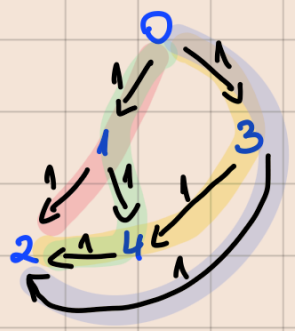
	node	total cost	neighb.	queue until now	add to queue
it 1	0	0	{1,2}	[ ]	(1,1,0) (1,2,0)
it 2	1	1	{0,2,3}	[(1,2,0)]	(1,2,1) (2,3,1)
it 3	2	2	{0,1,3,4}	[(1,2,1),(2,3,1)]	(2,3,2) (3,4,2)
it 4:	3	4	{1,2,4,5}	[(2,3,2),(3,4,2)]	(3,4,3) (4,5,3)
it 5	4	7	{2,3,5}	[(3,4,3),(4,5,3)]	(4,5,4)
it 6	5	11	{3,4}	[(4,5,4)]	-

bonus 1 :



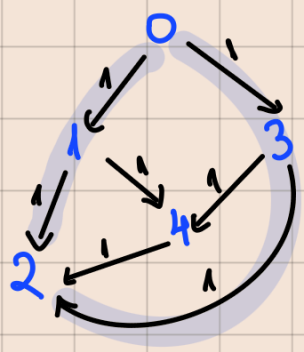
preorder:	0	1	2	3	7	8	4	5	6
postorder:	1	7	8	3	4	6	5	2	0
inorder:	1	0	7	3	8	2	5	4	6

bonus 2 :



paths: 0 → 1 → 2
0 → 1 → 4 → 2
0 → 3 → 4 → 2
0 → 3 → 2

bonus 3 :



cost : 2
paths: 0 → 1 → 2
0 → 3 → 2